

SkyTrack GNSS Explorer — Detailed Project Specification

1. Project Overview

Project name: SkyTrack GNSS Explorer

Target platforms: Windows 10/11 (64-bit .exe) and Android 10+ (.apk)

Application framework: Flutter and Dart

Planned duration: 25 working days

Primary purpose: Receive positioning data from a GNSS source, parse the incoming data, display live navigation metrics, and store timestamped logs locally.

The Windows and Android applications will share the same Flutter user interface, GNSS parser, state management, data models, simulation engine, and logging rules. Platform-specific adapters will be used for Bluetooth/serial connectivity, permissions, storage paths, application lifecycle, and distribution.

Important scope decision: External GNSS hardware must expose one of the protocols listed in Section 5. "Bluetooth support" alone is not sufficient because BLE GATT, Bluetooth Classic SPP, and Windows virtual COM ports are not interchangeable.

2. Project Goals

The application will:

1. Receive GNSS data from a supported live source or a sample file.
2. Parse supported NMEA 0183 sentences without blocking the user interface.
3. Display the most recent valid position, speed, altitude, heading, fix status, satellite count, and data age.
4. Record normalized GNSS observations in human-readable log files.
5. Rotate log files at a user-selected interval without losing records.
6. Recover cleanly from Bluetooth disconnections, invalid sentences, denied permissions, and unavailable storage.
7. Produce separately tested Windows and Android release packages.

The application is a data consumer and visualizer. It will not perform raw RF signal acquisition, satellite tracking, pseudorange calculation, or position solving as GNSS-SDR does. GNSS-SDR may be used only as a protocol/data-flow reference. The connected receiver is responsible for calculating the navigation solution and outputting NMEA data.

3. Platform Deliverables

3.1 Windows deliverable (.exe)

The Windows release will be a signed-ready x64 Flutter desktop build delivered as an installer or a ZIP package containing the executable and all required Flutter runtime files. A single standalone .exe must not be promised unless an additional packaging technology is selected, because a standard Flutter Windows build includes supporting DLLs and a data directory. The Windows build will primarily connect to external receivers through a serial COM port. A Bluetooth Classic receiver is expected to be paired in Windows Settings first; Windows then exposes it as a virtual COM port. USB GNSS receivers that expose a COM port may use the same adapter.

Windows-specific behavior:

- Enumerate available COM ports and show port name, description, and manufacturer when provided by the OS.
- Allow the user to choose baud rate: 4800, 9600, 38400, 57600, or 115200; default 9600.
- Use serial configuration 8 data bits, no parity, 1 stop bit, no flow control unless a receiver profile specifies otherwise.
- Remember the last selected port and baud rate, but require reconnection if the port is no longer available.
- Detect unplugged devices, closed ports, and read failures and move the UI to Disconnected without crashing.
- Save logs by default under %USERPROFILE%\Documents\SkyTrack\Logs or an app-specific user-data directory. The user may select another writable folder.
- Use the normal Windows file picker for simulation files and log export.
- Continue receiving and logging while the window is minimized. Logging stops when the process exits unless a separate Windows service is added to scope.
- Display a system-tray/background service only if it is approved as an additional requirement.

3.2 Android deliverable (.apk)

The Android release will be a release-mode APK supporting Android 10 and later. An Android App Bundle (.aab) can also be produced if Play Store distribution is required. The APK must be signed with a release keystore for production distribution; a debug-signed APK is for testing only.

Android-specific behavior:

- Support the phone/tablet's internal location provider through Android location services.
- Support an external receiver over Bluetooth Low Energy only when the receiver exposes a documented GATT service and characteristic that streams NMEA bytes.
- If Android Bluetooth Classic SPP support is required, add and test a separate SPP plugin/native adapter. flutter_blue_plus supports BLE and must not be treated as Bluetooth Classic SPP support.

- Request Bluetooth and location permissions at runtime and explain why each permission is needed.
- Use Android's Storage Access Framework for user-selected simulation files and exported logs.
- Store private working logs in the application's files directory. Files must not depend on broad storage permission.
- Handle activity pause/resume, Bluetooth being disabled, Android process termination, and device sleep.
- Foreground-only acquisition is included by default. Continuous logging while the app is not visible requires an Android foreground service with a persistent notification and must be explicitly enabled in scope.
- Show a warning when battery optimization or background restrictions may interrupt an approved foreground logging service.

4. Windows .exe vs Android .apk — Required Differences

Area	Windows .exe	Android .apk
Primary live transport	Serial COM port, including Bluetooth/USB virtual COM	Internal Android location or external BLE GATT; Classic SPP only with a separate adapter
Device discovery	Enumerate OS COM ports	Scan nearby BLE devices and filter by service UUID/name
Pairing	Normally completed in Windows Settings before COM connection	BLE may connect in-app; bonded/encrypted devices may trigger Android pairing UI
Connection settings	COM port and baud rate are user-selectable	BLE service UUID, characteristic UUID, notification setup, and MTU are defined by receiver

Area	Windows .exe	Android .apk
		profile
Permissions	Usually no runtime permission dialog for serial ports; folder access depends on OS policy	Runtime Bluetooth/location permissions vary by Android API level
Internal GPS	Not guaranteed and excluded unless Windows Location API support is separately implemented	Included through Android location services
File access	Native path-based picker and user-readable filesystem paths	URI-based Storage Access Framework; app-private paths may not be directly browsable
Default log storage	Documents/app-data folder	App-private files directory with explicit Export/Share action
Background behavior	Works while minimized; stops when process exits	Foreground only by default; persistent background logging requires a foreground service
Installation	Installer/ZIP with .exe, DLLs, and data assets	Signed APK installed by sideloading/MDM; Play distribution normally uses AAB
Updates	New installer/package replaces existing build	New APK must use the same package ID and signing key and a higher

Area	Windows .exe	Android .apk
		version code
Navigation/UI	Keyboard/mouse, resizable window, wider layouts	Touch controls, narrow portrait layout, Android back navigation
Testing hardware	Windows 10/11 x64 and actual COM-port receiver	At least two Android API levels and actual BLE/SPP receiver matching approved protocol

5. Supported Data Sources and Protocol Contract

5.1 Android internal location

The Android internal source will provide latitude, longitude, altitude when available, speed, heading, horizontal accuracy, provider timestamp, and fix age. Satellite count may be unavailable through the selected Flutter location plugin. If satellite-level status is mandatory, Android GNSS status APIs must be integrated through a compatible plugin or native Kotlin platform channel.

5.2 Windows serial/COM source

The serial adapter will read a continuous byte stream, decode ASCII NMEA text, preserve partial lines between reads, and split records on `\r\n`, `\n`, or `\r`. The adapter must tolerate multiple sentences arriving in one chunk and one sentence arriving across multiple chunks.

5.3 Android BLE source

BLE support requires a receiver profile containing:

- advertised name or service UUID used for scan filtering;
- GATT service UUID;
- notify/read characteristic UUID carrying GNSS data;
- optional write characteristic UUID for receiver commands;
- encoding, expected line delimiter, and maximum message rate;
- whether pairing/bonding is required.

The app will subscribe to characteristic notifications and reassemble notification packets into

complete NMEA lines. BLE packet boundaries must not be assumed to match NMEA sentence boundaries.

5.4 Android Bluetooth Classic SPP source

If selected, the SPP adapter will connect to the receiver's RFCOMM serial service and expose the bytes through the same internal stream interface as COM and BLE. Hardware model, Android version, pairing flow, and reconnect behavior must be validated with the actual receiver. This adapter is a separate implementation item and cannot be delivered using flutter_blue_plus alone.

5.5 Simulation source

The simulation source will accept .txt, .log, or .nmea files containing one NMEA sentence per line. Playback controls will include Start, Pause, Stop, speed (0.5x, 1x, 2x, 5x), and optional loop. At 1x, records will use timestamps from the file when available; otherwise a configurable delay, default one second, will be used.

All source adapters will implement a common contract similar to:

```
abstract interface class GnssSource {  
    Stream<Uint8List> get byteStream;  
    Stream<SourceStatus> get statusStream;  
    Future<void> connect(SourceConfig config);  
    Future<void> disconnect();  
}
```

6. NMEA Parsing Requirements

Initial support will cover NMEA 0183 ASCII sentences with standard \$ or ! prefixes.

Version-specific receiver extensions are excluded until sample data is provided.

Supported sentences:

- **GGA:** UTC time, latitude, longitude, fix quality, satellites used, HDOP, altitude, and geoid separation.
- **RMC:** UTC date/time, validity, latitude, longitude, speed over ground, and course over ground.
- **GSA:** fix mode, satellites used, PDOP, HDOP, and VDOP.
- **GSV:** satellites in view and multi-sentence satellite information.
- **VTG:** course and speed over ground.
- **GLL:** position, UTC time, and validity, used as a fallback.

The parser will accept common talker IDs such as GP, GN, GL, GA, GB, and BD. It will not hard-code only \$GP... sentences.

Validation rules:

1. Maximum accepted line length: 1,024 bytes unless a receiver extension requires more.
2. Verify XOR checksum when *HH is present.
3. Reject malformed coordinates, impossible latitude/longitude ranges, and non-numeric required fields.
4. Do not replace a valid displayed fix with a sentence marked invalid.
5. Count and expose received, parsed, checksum-failed, malformed, and unsupported sentences for diagnostics.
6. Normalize knots to kilometres per hour using $\text{km/h} = \text{knots} \times 1.852$.
7. Normalize heading to $0 \leq \text{heading} < 360$ degrees.
8. Store timestamps in UTC ISO 8601 and convert to local time only for display.
9. Combine RMC date with GGA time when creating a full GNSS timestamp.
10. Mark data as stale when no valid update is received for five seconds; make this threshold configurable internally.

The normalized position model will contain at least:

```
received_at_utc, gnss_time_utc, source, latitude_deg, longitude_deg,
altitude_m, speed_kmh, heading_deg, fix_valid, fix_quality,
satellites_used, satellites_in_view, hdop, vdop, pdop,
raw_sentence_type
```

7. Application Architecture

The codebase will use feature-oriented layers:

```
lib/
  core/          shared errors, time, utilities, configuration
  domain/        GNSS models and source/logging interfaces
  data/
    sources/     simulation and internal-location adapters
    parser/      NMEA framing, checksum, and sentence parsers
    logging/     buffering, serialization, and rotation
  platform/
    android/     BLE, optional SPP, permissions, storage
    windows/     serial COM and Windows storage
  features/
    dashboard/
    source_selection/
    settings/
    diagnostics/
```

Riverpod is recommended for dependency injection, source status, parsed position state, and lifecycle-controlled stream subscriptions. Only one live source may be active at a time.

Changing sources must disconnect and dispose the previous source before connecting the next one.

Data flow:

```
GNSS source -> byte framing -> NMEA validation/parser -> normalized fix stream
                                                    -> dashboard state
                                                    -> logger buffer
                                                    -> diagnostics counters
```

Parsing and file writes must be asynchronous. If measured input volume causes UI frame drops, parsing will be moved to a Dart isolate. The application must use bounded buffers so that a stalled disk or UI consumer cannot cause unlimited memory growth.

8. Functional Requirements

8.1 Dashboard tab

Display:

- latitude and longitude to at least six decimal places;
- altitude in metres;
- speed in km/h;
- heading in degrees and optional compass label;
- fix state (No Fix, 2D, 3D, DGPS/RTK when reported);
- satellites used and satellites in view;
- HDOP when available;
- source name and transport;
- connection state;
- time of last valid fix and live data age.

Connection states will be: Idle, Permission Required, Scanning, Connecting, Connected, Receiving, Reconnecting, Reading File, Disconnected, and Error.

No-data fields will show —, not zero, because zero is a valid value for several navigation measurements.

8.2 Source Selection tab

- Select Internal GPS, External Receiver, or Simulation.
- Android BLE list: device name, address/identifier, signal strength, and compatible/incompatible status.
- Windows serial list: COM name, description, and baud-rate selector.
- Scan refresh and stop controls.
- Connect/disconnect control with timeout and actionable errors.
- Simulation file picker and playback controls.

- Persist the last source type and safe settings. Do not auto-connect on startup unless the user enables it.

8.3 Settings tab

- Enable/disable logging.
- Log sampling interval: 1 s, 5 s, or 10 s.
- File rotation interval: 30 minutes, 1 hour, or daily.
- Log output format: normalized CSV is required; optional raw NMEA file may be enabled separately.
- Display current storage location and available export/open-folder action appropriate to the platform.
- Choose Windows output folder or Android export destination.
- Enable/disable automatic reconnect, default enabled with a limited retry policy.
- Clear diagnostics counters.

8.4 Diagnostics view

For support and testing, expose:

- app version/build number and platform;
- source and connection parameters excluding secrets;
- bytes received;
- NMEA records received/parsed/rejected;
- checksum and malformed-record counts;
- last error and timestamp;
- current log filename and records written;
- permission state.

9. Logging, Rotation, and Recovery

9.1 Normalized CSV format

Each file will begin with a fixed CSV header. Fields containing commas or quotes must use RFC 4180-compatible escaping. Missing values will be empty, not zero.

Example:

```
received_at_utc,gNSS_time_utc,source,latitude_deg,longitude_deg,altitude_m,speed_kmh,heading_deg,fix_valid,fix_quality,satellites_used,satellites_in_view,hdop
2026-09-06T10:30:01.220Z,2026-09-06T10:30:01.000Z,windows_com,19.076090,72.877426,14.2,7.41,188.4,true,gps_fix,10,15,0.8
```

Raw NMEA logging, if enabled, will preserve the original sentence and add a separate receive timestamp. It will not be mixed with normalized CSV rows.

9.2 Sampling and rotation

The write interval controls normalized sample frequency, not serial/BLE read frequency. The parser will continue processing all incoming sentences while the logger records the latest valid normalized fix at each sampling tick.

File names:

```
SkyTrack_<source>_<UTC-start>_<UTC-end>.csv  
SkyTrack_raw_<source>_<UTC-start>_<UTC-end>.nmea
```

Rotation procedure:

1. Write to a file ending in .partial.
2. Flush buffered records at least every five seconds and on graceful disconnect/exit.
3. At the rotation boundary, stop assigning new records to the old writer.
4. Flush and close the old file.
5. Rename it to its final timestamped name.
6. Open a new .partial file and write its header.

This is file rotation, not "merging." Existing rotated files will not be repeatedly reopened and combined. If a single combined export is required, it will be a separate user-triggered operation. On startup, the app will inspect its own log directory for .partial files. A non-empty recoverable file will be closed/renamed with a recovered suffix; it will not be silently deleted.

9.3 Storage estimates and limits

At one normalized row per second, expected storage is approximately 10–25 MB per day depending on populated columns. Raw NMEA logging can be substantially larger and depends on receiver output rate. The application will warn when writes fail or free space is insufficient. Automatic deletion of old logs is excluded unless a retention rule is added.

10. Permissions and Security

Android

The final manifest and runtime requests will depend on the minimum/target SDK and selected transports. Expected permissions include modern Bluetooth scan/connect permissions, location permission for internal GNSS and older Android BLE behavior, and foreground-service permissions only if background logging is approved.

The application will:

- request permissions only when the relevant feature is selected;
- handle denied, denied permanently, and service-disabled states;
- link users to system settings when permission is permanently denied;
- avoid broad "all files" storage access;
- avoid collecting or transmitting location data to a server.

Windows

The application will not require administrator privileges for normal operation. Logs and settings will be stored in user-writable locations. Code signing is strongly recommended to reduce SmartScreen warnings, but the signing certificate is supplied by the project owner and is not included in development cost unless specified.

Shared

All location data remains on the device. No cloud synchronization, account system, analytics, advertising SDK, or remote telemetry is included. Sensitive receiver identifiers must not be written to diagnostic exports unless the user explicitly chooses to include them.

11. Reliability and Performance Requirements

- Dashboard values should update within 500 ms of a complete valid NMEA sentence under normal conditions.
- The parser should sustain at least 100 NMEA sentences per second in a synthetic test without unbounded memory growth.
- UI should target 60 frames per second and remain responsive during scanning, parsing, and logging.
- Connection attempt timeout: 15 seconds unless the platform API returns earlier.
- Automatic reconnect: up to five attempts with delays of 1, 2, 4, 8, and 16 seconds; then require manual retry.
- Duplicate subscriptions and concurrent connections must be prevented.
- A bad sentence must affect only that sentence, not terminate the stream.
- Logging failure must show an error and disable logging safely while live visualization continues.
- App settings will be versioned so future releases can migrate stored preferences.

12. Testing and Acceptance Criteria

Shared tests

- Unit tests for checksum validation, coordinate conversion, every supported sentence type, malformed input, multi-constellation talker IDs, unit conversions, and timestamp assembly.
- Stream-framing tests for split sentences, multiple sentences per packet, mixed line endings, noise before \$, and invalid UTF-8/ASCII bytes.
- Logging tests for CSV escaping, sampling interval, rotation boundary, crash recovery, failed writes, and no-fix behavior.
- Simulation tests using valid, invalid, delayed, and high-rate sample files.

Windows acceptance

1. A supported receiver appears in the COM list after OS pairing/connection.
2. The app connects using the configured baud rate and displays a valid position.
3. Unplugging/turning off the receiver changes status without crashing and triggers the retry policy.
4. Logs are written to the selected folder and can be opened by a normal desktop CSV/text tool.
5. The packaged release launches on a clean Windows 10 and Windows 11 x64 machine without a Flutter development environment.

Android acceptance

1. The app obtains and displays internal location after user permission and a valid device fix.
2. A specifically approved external receiver can be discovered, connected, and read using its documented BLE or SPP profile.
3. Permission denial, Bluetooth disabled, and permanent denial produce actionable UI states.
4. Logs survive normal app restart and can be exported through Android's document picker/share flow.
5. The signed release APK installs and launches on agreed physical devices and Android versions.
6. If background logging is included, an eight-hour screen-off test completes with a foreground notification and without unexplained logging gaps.

13. Proposed Technology Stack

- Flutter stable channel and Dart, pinned in project documentation.
- Riverpod for state management and dependency injection.
- flutter_libserialport or a validated equivalent for Windows serial COM communication.
- flutter_blue_plus for Android BLE only.
- A validated SPP plugin or custom Kotlin platform channel if Android Classic SPP is approved.
- A maintained location plugin for Android internal GNSS/location.
- path_provider for application-owned directories.
- Android Storage Access Framework-compatible file picker/export plugin.
- shared_preferences or an equivalent lightweight store for non-sensitive settings.
- package_info_plus and platform/device information packages for diagnostics.

All dependencies will be pinned to tested versions. Package selection will be confirmed against current Windows/Android support before implementation.

14. Revised 25-Day Plan

Phase	Days	Output
Discovery and hardware contract	1–2	Confirm receiver models, BLE/SPP/COM protocols, sample NMEA, Android versions, and delivery format
Architecture and project setup	3–4	Flutter targets, adapters/interfaces, CI/linting, data models, initial tests
Shared UI	5–8	Responsive Dashboard, Source Selection, Settings, and Diagnostics views
Parser and simulation	9–12	Stream framing, NMEA parser, normalized model, simulation playback, unit tests
Windows integration	13–15	COM discovery/configuration, streaming, errors, Windows file handling
Android integration	16–18	Internal location, BLE profile integration, permissions, Android storage
Logging and recovery	19–21	CSV/raw logging, sampling, rotation, export, partial-file recovery

Phase	Days	Output
End-to-end testing	22–24	Physical-device testing, reconnect/lifecycle tests, fixes, performance checks
Release and handoff	25	Windows package, signed/test APK, source, sample data, test report, user manual

The 25-day estimate assumes one known external receiver profile is supplied at project start, required hardware is available for testing, and Android Classic SPP/background service are either excluded or confirmed early. Supporting multiple undocumented receiver protocols may require additional time.

15. Final Deliverables

1. Windows x64 release package containing the executable and runtime files; optional installer if selected.
2. Signed Android release APK; optional AAB if Play Store delivery is selected.
3. Complete Flutter source code with platform adapters and build instructions.
4. Automated unit tests and a test-results/acceptance checklist.
5. Valid and intentionally invalid NMEA sample files for simulation and parser testing.
6. Configuration document listing supported receiver model(s), transport, UUIDs/ baud rates, and tested firmware.
7. User manual covering connection, permissions, simulation, logging, export, and common errors.
8. Known-limitations document.

16. Explicitly Out of Scope Unless Added

- Raw GNSS RF/sample processing or embedding GNSS-SDR.
- Receiver firmware updates or proprietary binary-protocol reverse engineering.
- Map tiles, turn-by-turn navigation, route planning, or geofencing.
- Cloud backup, synchronization, web dashboard, accounts, or remote telemetry.
- iOS, macOS, Linux, or web versions.
- Windows background service/system-tray mode.

- Android continuous background logging without a foreground service.
- Support for arbitrary Bluetooth devices without a documented/tested receiver profile.
- Play Store publication, Microsoft Store publication, or purchase of code-signing certificates.
- Automatic deletion/retention of old logs.

17. Inputs Required Before Development Starts

The project owner must provide or approve:

1. Exact GNSS receiver manufacturer, model, and firmware version.

Normal Bluetooth HC05 bluetooth receiver is there breakout board input is HC05. G

2. For Android: BLE GATT UUIDs or confirmation that the receiver uses Bluetooth Classic SPP.

3. For Windows: expected COM baud rate and serial configuration.

9600 standard and slave mode device HC05 serial UUID signal high low or receiving pulses uses standard receiving pulses

4. At least 15 minutes of representative NMEA output, including no-fix and valid-fix periods. Generate this using a standard data and make a csv – it has 4 main parameters- timestamp lat lon and altitude – basic parameters and more which you can use. Generate standard GNSS 100 samples output file make csv and I will forward and check. We will use virtual COM port in laptop and 80+ URT series 9600 + 0 + 0

5. Minimum Android version and target test devices.

Android version – we will use c-type converter not bluetooth module in C-type D+ D- and VCC so in apk we need no bluetooth and power from phone CC and blob and in windows we are using bluetooth instead of c-type as we dont have that

6. Whether Android logging must continue with the screen off/app in background.

Yes we should HAVE IT WORKING ON screen off ALSO. battery saver will be off and ask user for permission and that application will always run and disable battery saving mode

7. Whether raw NMEA, normalized CSV, or both are required.

Raw NMEA - GNSS records & MSM altitude, timestamp, lat and lon and we need normalised CSV we need not show raw NMEA and directly store line by line per second

8. Required Windows delivery format: ZIP, installer, or Microsoft Store package.

ZIP format or .exe will be fine

9. Release signing keys/certificates or approval to deliver unsigned test builds.

We can generate 50 keys for now and we can even hardcode it and it will fetch it from somewhere in v1 got it?